



APPLICATION DEVELOPMENT PRACTICES

Dr.Lê Hùng Tiến
VLU



APPLICATION DEVELOPMENT PRACTICES

Refactoring



Director's Overview

- What is Refactoring?
- Why Refactor?
- Reasons Not to Refactor?
- How to Refactor
- Refactoring in eclipse



What is Refactoring?

- "Refactoring" source code means improving it without changing what it does
- Refactoring does NOT:
 - Fix defects
 - Add new functionality
- The goal of refactoring is to:
 - Improve the understandability of the code
 - Improve the structure of the code
 - Remove unnecessary code



What is Refactoring?

- Refactoring is a disciplined technique
 - Each refactor should be small
 - So it is less likely to go wrong
 - The system is kept fully working after each small refactoring
 - Reducing the chances that a system can get seriously broken during the restructuring
 - Increasing the need for automated unit test
- Refactoring can produce a significant benefit over time



Why Refactor?

- Because software evolves over time
 - During development
 - Original developers involved
 - During maintenance
 - Different developers likely to be involved
 - Original intent of developers has been forgotten
- Because of “code smells”
 - Smells are heuristics that can indicate when and what to refactor
 - Smells are indicators, not causes
- Failure to perform refactoring can result in accumulating technical debt



Is Your Software's Evolution ...

- A planned-for opportunity that is improving the internal quality of the software?
- A haphazard activity that is continually degrading the product's quality?

Is Quality An Accident?



Some Reasons to Refactor - 1

- Duplicate code
 - Must make changes in multiple places
- Routine that is too long
 - Routines should do one thing well
- Loops are too long or too deeply nested
 - Convert loop content into routines?
- Poor cohesion
 - Methods that implement a single function are described as having high cohesion



Some Reasons to Refactor - 2

- Class interface with an inconsistent level of abstraction
 - May want to recapture interface integrity
- Parameter list with too many parameters
 - Well-factored programs tend to have many small, well-defined routines that don't require large parameter lists
- Class changes are compartmentalized
 - A class has too many responsibilities



Some Reasons to Refactor - 3

- Parallel modifications to multiple classes
 - Should classes be rearranged so that changes affect only one class
- Inheritance hierarchies have to be modified in parallel
 - Making a subclass of one class every time you make a subclass of another class is another form of parallel modification
- Related data items that are used together are not organized into classes



Ex: Data Level Refactoring

- Replace a magic number with a named constant
 - Use a named constant (e.g. "Pi") instead of a literal (e.g. "3.14")
- Rename a variable with a clearer or more informative name
 - Replace "name" with "accountname"



Ex: Statement Level Refactoring

- Consolidate fragments that are duplicated within different parts of a conditional
 - If same lines of code are repeated in a conditional
- Replace conditionals with polymorphism (especially with repeated case statements)
- Create and use null objects instead of testing for null values
 - Move null checking code away from the client and into the class



Ex: Routine Level Refactoring

- Extract a routine
 - Remove inline code from one routine, and turn it into its own routine
- Convert a long routine to a class
 - If a routine is too long then maybe it should be its own class
- Substitute a simple algorithm for a complex algorithm
 - Simplify
- Remove a parameter
 - If the parameter is no longer used



Ex: Class Implementation Refactor

- Extract specialized code into a subclass
 - If a class has code that's used by only a subset of its instances, move that specialized code into its own subclass
- Combine similar code into a superclass
 - If two subclasses have similar code, combine that code and move it into the superclass



Ex: Class Interface Refactoring

- Convert one class to two
 - If a class has more than one distinct area of responsibility
- Eliminate a class
 - If the class isn't doing much
- Encapsulate an exposed member variable
 - Change the data to private and expose the data's value through a routine instead



Ex: System Level Refactoring

- Duplicate data you can't control
 - If you have multiple sources that must access data, then move the data to its own class and have all sources treat that class as the definitive source of the data



Reasons NOT to Refactor

- Refactoring is NOT defect fixing, adding functionality or modifying the design
 - Do these types of maintenance efforts separately
- Sometimes code is so bad it needs to be rewritten



How to Refactor - 1

- Make sure you can get back to where you started
 - SCM system
 - Backups
- Keep refactoring as small as you can
- Do one refactoring at a time
- Make a list of refactoring steps you intend to take
- Log additional refactoring ideas/needs that you encounter

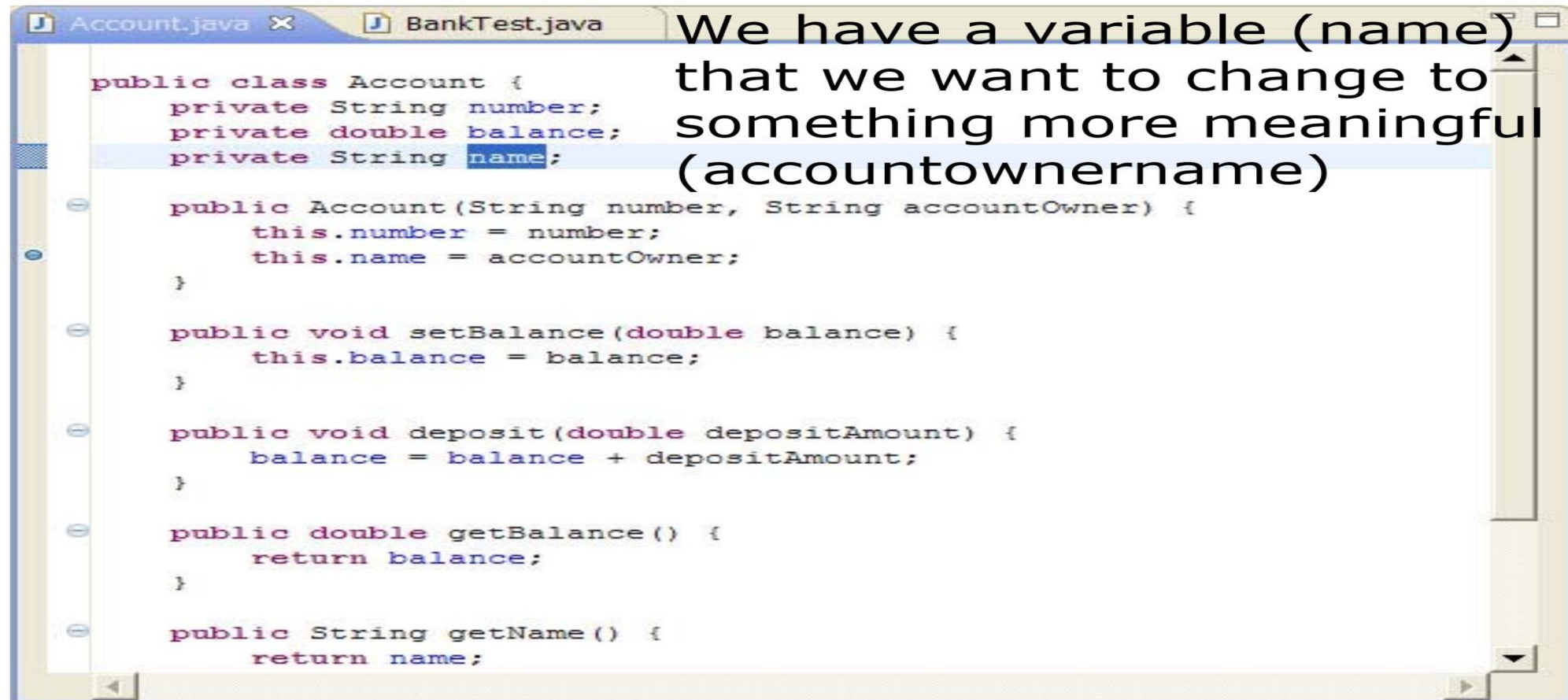


How to Refactor - 2

- Make frequent checkpoints
- Execute existing unit tests
- Add new unit tests
- Recognize that different refactoring efforts include different levels of risk
 - Err on the side of caution
 - Peer review your refactoring changes



Refactoring in eclipse



We have a variable (name) that we want to change to something more meaningful (accountownername)

```
public class Account {  
    private String number;  
    private double balance;  
    private String name;  
  
    public Account(String number, String accountOwner) {  
        this.number = number;  
        this.name = accountOwner;  
    }  
  
    public void setBalance(double balance) {  
        this.balance = balance;  
    }  
  
    public void deposit(double depositAmount) {  
        balance = balance + depositAmount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```




Refactoring Menu in eclipse

Right click on
the variable,
select
Refactor and
then Rename

Note the other
refactoring
tools available

Open Declaration	F3
Open Type Hierarchy	F4
Open Call Hierarchy	Ctrl+Alt+H
Show in Breadcrumb	Alt+Shift+B
Quick Outline	Ctrl+O
Quick Type Hierarchy	Ctrl+T
Show In	Alt+Shift+W
▶	
Cut	Ctrl+X
Copy	Ctrl+C
Copy Qualified Name	
Paste	Ctrl+V
▶	
Quick Fix	Ctrl+1
Source	Alt+Shift+S
▶	
Refactor	Alt+Shift+T
Surround With	Alt+Shift+Z
Local History	
▶	
References	
Declarations	
▶	
Add to Snippets...	
▶	
Run As	
Debug As	
Profile As	
Validate	
Team	
▶	

Rename...	Alt+Shift+R
Move...	Alt+Shift+V
▶	
Extract Interface...	
Extract Superclass...	
Use Supertype Where Possible...	
Pull Up...	
Push Down...	
▶	
Extract Class...	
▶	
Encapsulate Field...	
▶	
Generalize Declared Type...	



Refactoring Menu in eclipse

Enter the new name for the variable, then press Enter

The screenshot shows the Eclipse IDE with two tabs: 'Account.java' and 'BankTest.java'. The 'Account.java' file is open, and the variable 'name' in the 'private String name;' line is selected. A context menu is visible over the variable, with the option 'Enter new name, press Enter to refactor' highlighted. The code in the background is as follows:

```
public class Account {  
    private String number;  
    private double balance;  
    private String name;  
  
    public Account(  
        this.number = number;  
        this.name = accountname;  
    }  
  
    public void setBalance(double balance) {  
        this.balance = balance;  
    }  
  
    public void deposit(double depositAmount) {  
        balance = balance + depositAmount;  
    }  
  
    public double getBalance() {
```



Refactoring Menu in eclipse

Note that eclipse has found everywhere that the variable is used and changed it there for me

```
public class Account {  
    private String number;  
    private double balance;  
    private String accountownername;  
  
    public Account(  
        this.number = number;  
        this.accountownername = accountname;  
    ) {  
  
    }  
  
    public void setBalance(double balance) {  
        this.balance = balance;  
    }  
  
    public void deposit(double depositAmount) {  
        balance = balance + depositAmount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public String getName() {  
        return accountownername;  
    }  
}
```



Summary

- Refactoring does NOT change the software's functionality
- Refactoring is just good programming
- Be careful and safe when you refactor
- The refactoring lists in this presentation are not complete. For complete lists:
 - Steve McConnell, *Code Complete*, 2nd edition. Chapter 24 (cc2e.com)
 - Martin Fowler, *Refactoring: Improving the Design of Existing Code*